Experiment 2: Exploratory Data Analysis on the Titanic Dataset

Aim: To perform descriptive statistical analysis and data visualization on the Titanic dataset using Python libraries such as Pandas, NumPy, Matplotlib, and Seaborn.

1. Import Required Libraries

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

2. Load the Dataset

In [2]:
```
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv"

df = pd.read_csv(url)

df.head()
```

Out[2]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_m
0	0	3	male	22.0	1	0	7.2500	S	Third	man	T
1	1	1	female	38.0	1	0	71.2833	C	First	woman	Fa
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	Fa
3	1	1	female	35.0	1	0	53.1000	S	First	woman	Fa
4	0	3	male	35.0	0	0	8.0500	S	Third	man	T

3. Dataset Overview

3.1 Dataset Information

In [3]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   survived     891 non-null    int64
 1   pclass       891 non-null    int64
 2   sex          891 non-null    str
 3   age          714 non-null    float64
 4   sibsp        891 non-null    int64
 5   parch        891 non-null    int64
 6   fare         891 non-null    float64
 7   embarked     889 non-null    str
 8   class        891 non-null    str
 9   who          891 non-null    str
 10  adult_male   891 non-null    bool
 11  deck         203 non-null    str
 12  embark_town  889 non-null    str
 13  alive        891 non-null    str
 14  alone        891 non-null    bool
dtypes: bool(2), float64(2), int64(4), str(7)
memory usage: 92.4 KB
```

3.2 Data Types and Missing Values

In [4]:
```
print("Data Types:")
print(df.dtypes)

print("\nMissing Values:")
print(df.isnull().sum())
```

```
Data Types:
survived         int64
pclass           int64
sex                str
age            float64
sibsp            int64
parch            int64
fare           float64
embarked           str
class              str
who                str
adult_male        bool
deck               str
embark_town        str
alive              str
alone             bool
dtype: object

Missing Values:
survived         0
pclass           0
sex              0
age            177
sibsp            0
parch            0
fare             0
embarked         2
class            0
who              0
adult_male       0
deck           688
embark_town      2
alive            0
alone            0
dtype: int64
```

4. Descriptive Statistics

4.1 Summary Statistics (Mean, Std, Min, Max, Quartiles)

In [5]:
```
df.describe()
```

Out[5]:

	survived	pclass	age	sibsp	parch	fare
count	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

4.2 Median

In [6]:
```
df.median(numeric_only=True)
```

Out[6]:
```
survived       0.0000
pclass         3.0000
age           28.0000
sibsp          0.0000
parch          0.0000
fare          14.4542
adult_male     1.0000
alone          1.0000
dtype: float64
```

4.3 Mode

In [7]:
```
df.mode(numeric_only=True).iloc[0]
```

Out[7]:
```
survived         0
pclass           3
age           24.0
sibsp            0
parch            0
fare          8.05
adult_male    True
alone         True
Name: 0, dtype: object
```

4.4 Variance

In [8]:
```
df.var(numeric_only=True)
```

Out[8]:
```
survived         0.236772
pclass           0.699015
age            211.019125
sibsp            1.216043
parch            0.649728
fare          2469.436846
adult_male       0.239723
alone            0.239723
dtype: float64
```

4.5 Standard Deviation

In [9]:
```
df.std(numeric_only=True)
```

Out[9]:
```
survived       0.486592
pclass         0.836071
age           14.526497
sibsp          1.102743
parch          0.806057
fare          49.693429
adult_male     0.489615
alone          0.489615
dtype: float64
```

5. Frequency Analysis of Categorical Variables

In [10]:
```
print("Sex:")
print(df["sex"].value_counts())

print("\nPassenger Class:")
print(df["class"].value_counts())

print("\nSurvival:")
print(df["survived"].value_counts())
```

```
Sex:
sex
male      577
female    314
Name: count, dtype: int64

Passenger Class:
class
Third     491
First     216
Second    184
Name: count, dtype: int64

Survival:
survived
0    549
1    342
Name: count, dtype: int64
```

6. Correlation Analysis

6.1 Correlation Matrix

In [11]:
```
correlation = df.corr(numeric_only=True)

correlation
```

Out[11]:

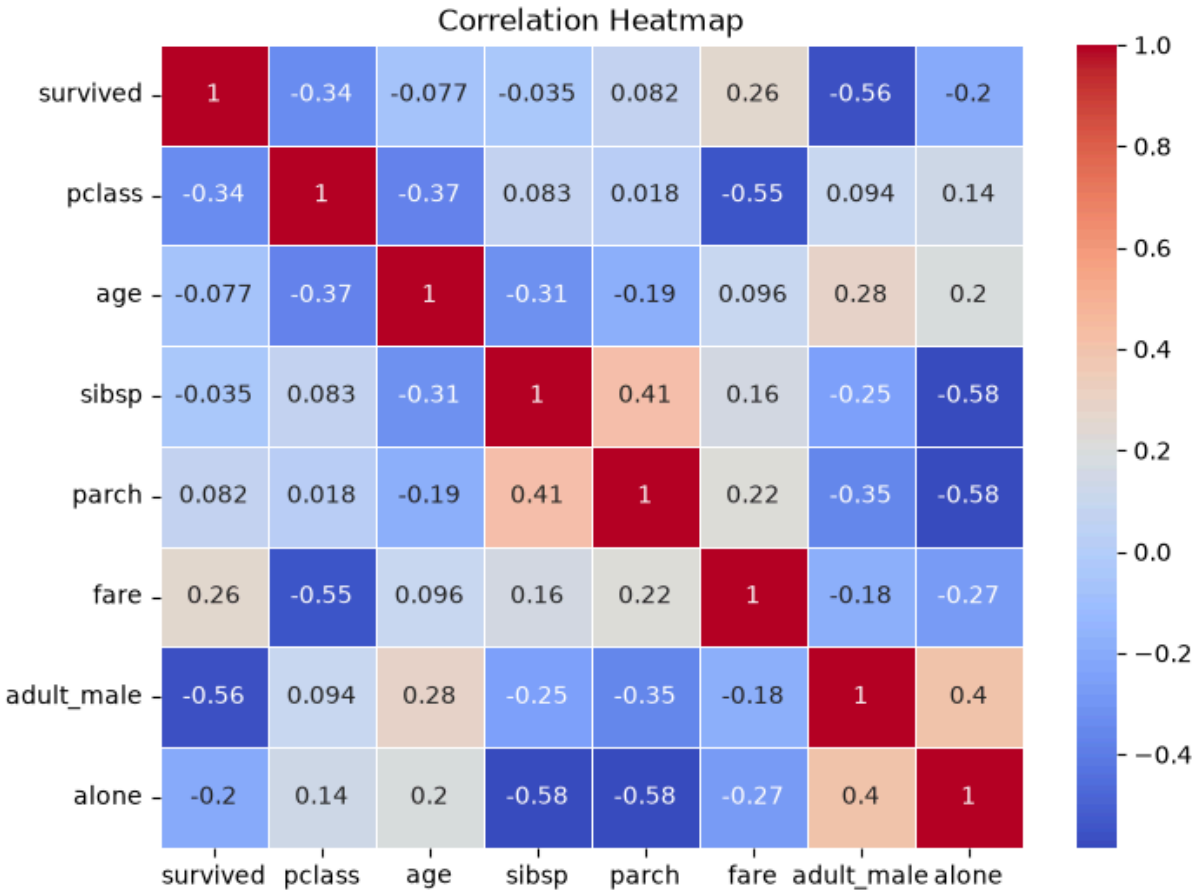
	survived	pclass	age	sibsp	parch	fare	adult_male	
survived	1.000000	-0.338481	-0.077221	-0.035322	0.081629	0.257307	-0.557080	-0
pclass	-0.338481	1.000000	-0.369226	0.083081	0.018443	-0.549500	0.094035	0
age	-0.077221	-0.369226	1.000000	-0.308247	-0.189119	0.096067	0.280328	0
sibsp	-0.035322	0.083081	-0.308247	1.000000	0.414838	0.159651	-0.253586	-0
parch	0.081629	0.018443	-0.189119	0.414838	1.000000	0.216225	-0.349943	-0
fare	0.257307	-0.549500	0.096067	0.159651	0.216225	1.000000	-0.182024	-0
adult_male	-0.557080	0.094035	0.280328	-0.253586	-0.349943	-0.182024	1.000000	0
alone	-0.203367	0.135207	0.198270	-0.584471	-0.583398	-0.271832	0.404744	1

6.2 Correlation Heatmap

In [12]:
```
plt.figure(figsize=(8, 6))

sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm",
    linewidths=0.5
)

plt.title("Correlation Heatmap")
plt.show()
```



7. Data Visualization

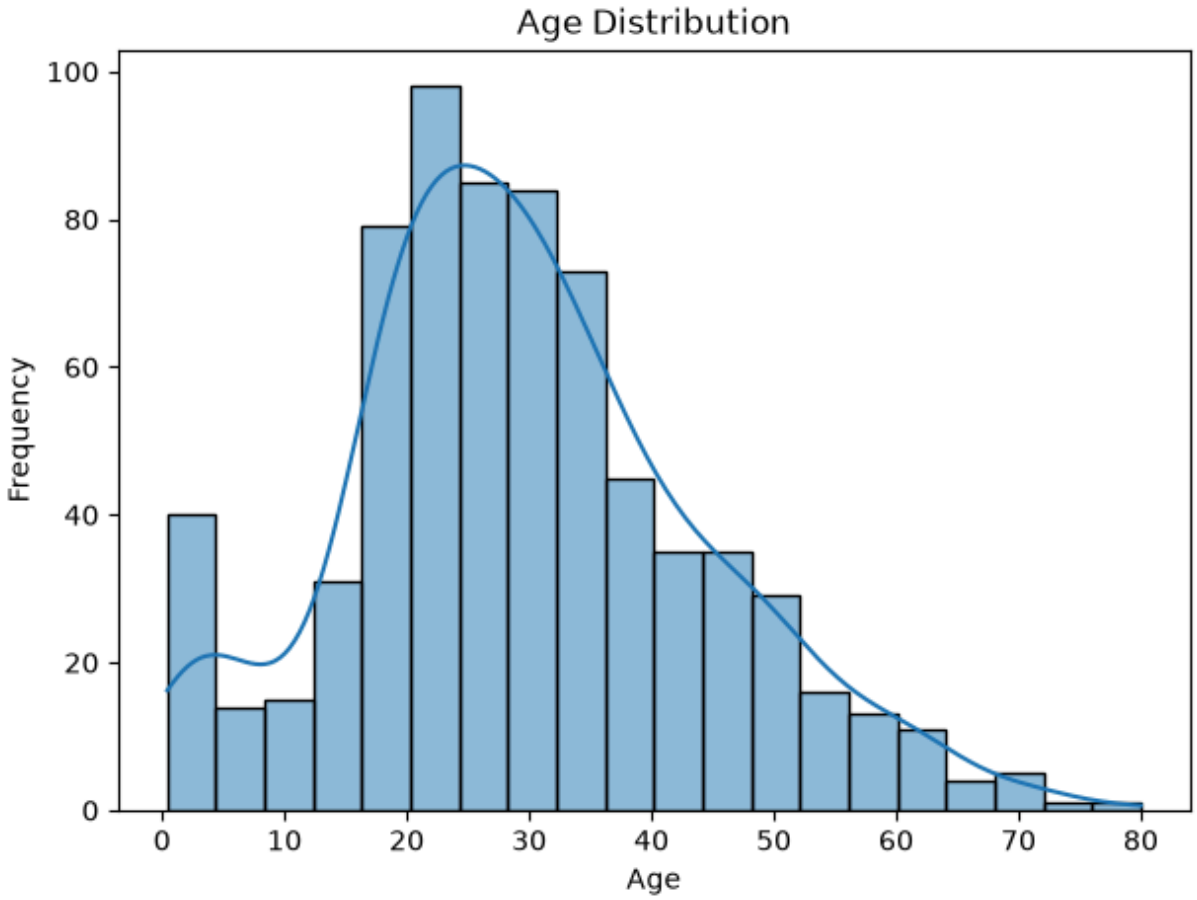
7.1 Age Distribution

In [13]:
```
plt.figure(figsize=(7, 5))

sns.histplot(df["age"].dropna(), kde=True)

plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")

plt.show()
```



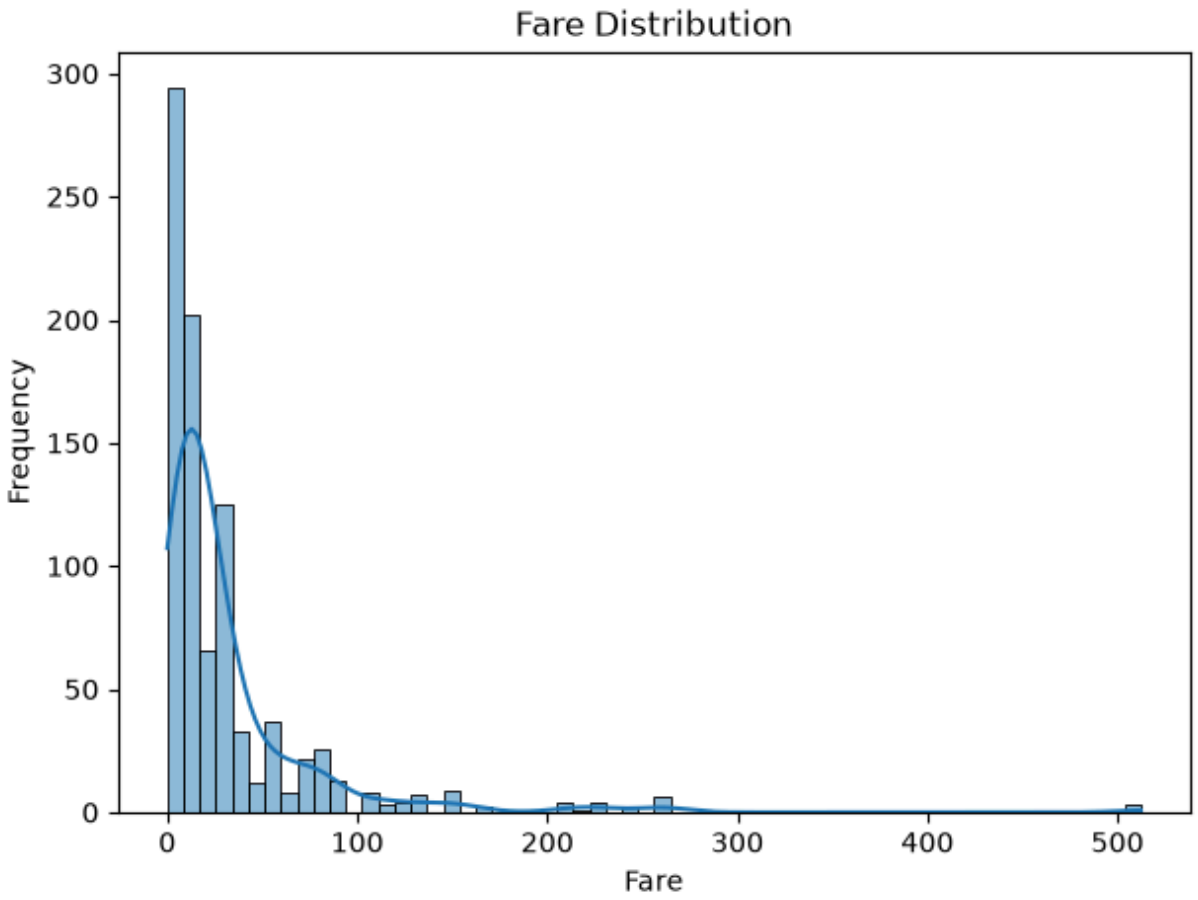
7.2 Fare Distribution

In [14]:
```
plt.figure(figsize=(7, 5))

sns.histplot(df["fare"], kde=True)

plt.title("Fare Distribution")
plt.xlabel("Fare")
plt.ylabel("Frequency")

plt.show()
```



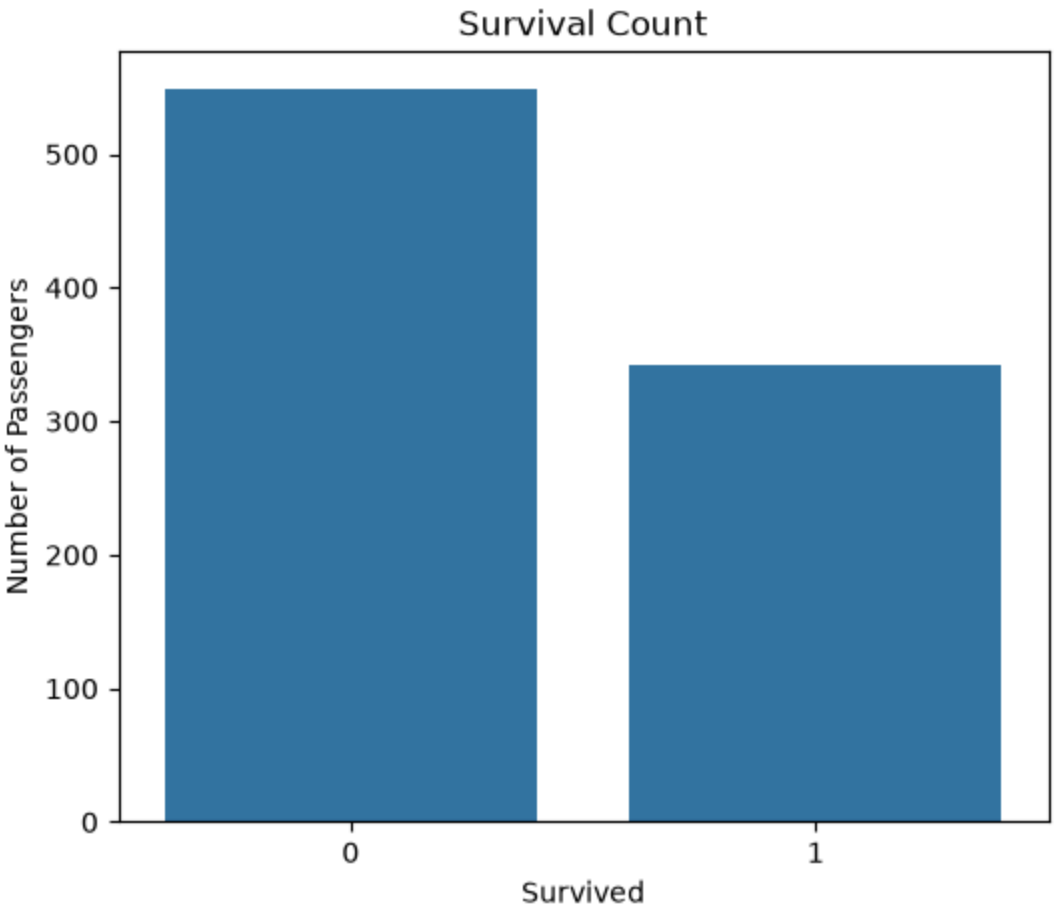
7.3 Survival Count

In [15]:
```
plt.figure(figsize=(6, 5))

sns.countplot(data=df, x="survived")

plt.title("Survival Count")
plt.xlabel("Survived")
plt.ylabel("Number of Passengers")

plt.show()
```



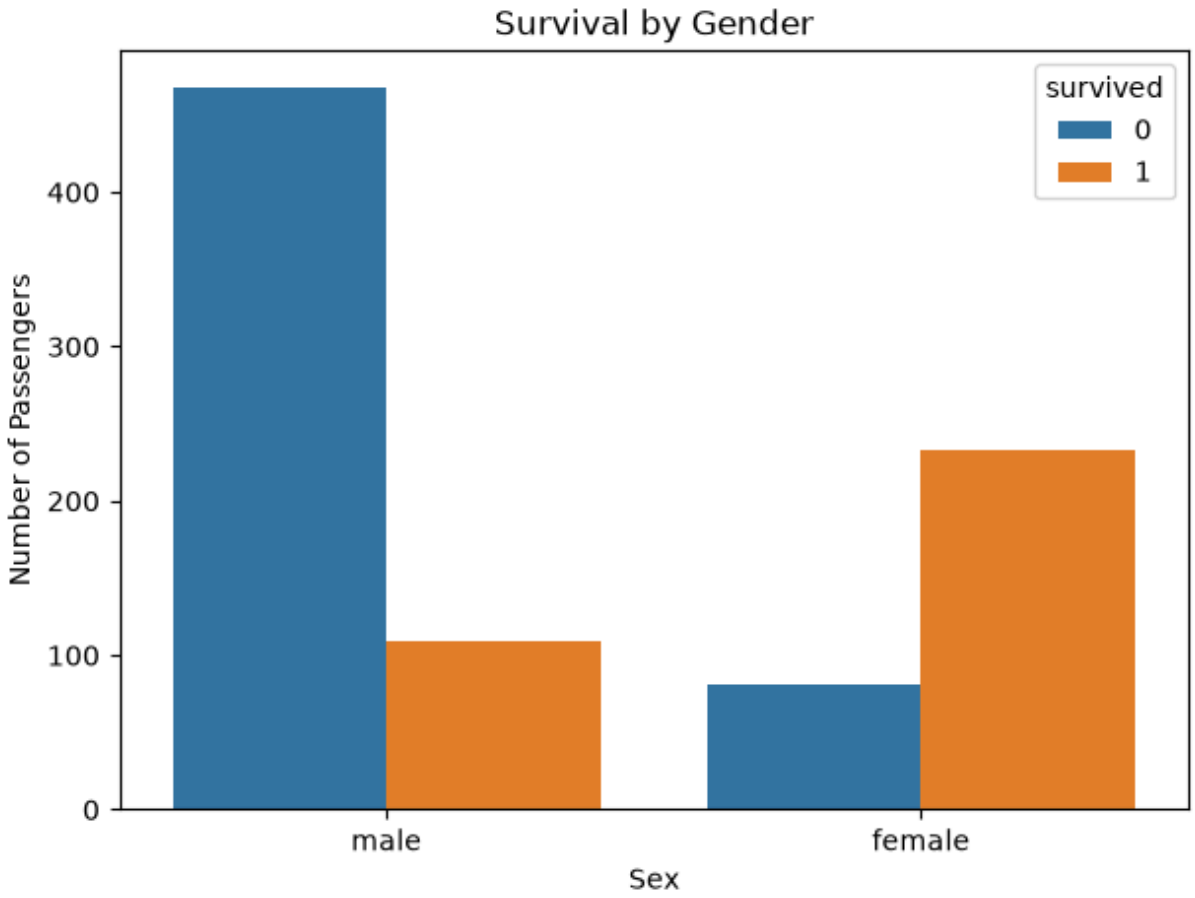
7.4 Survival by Gender

In [16]:
```
plt.figure(figsize=(7, 5))

sns.countplot(data=df, x="sex", hue="survived")

plt.title("Survival by Gender")
plt.xlabel("Sex")
plt.ylabel("Number of Passengers")

plt.show()
```



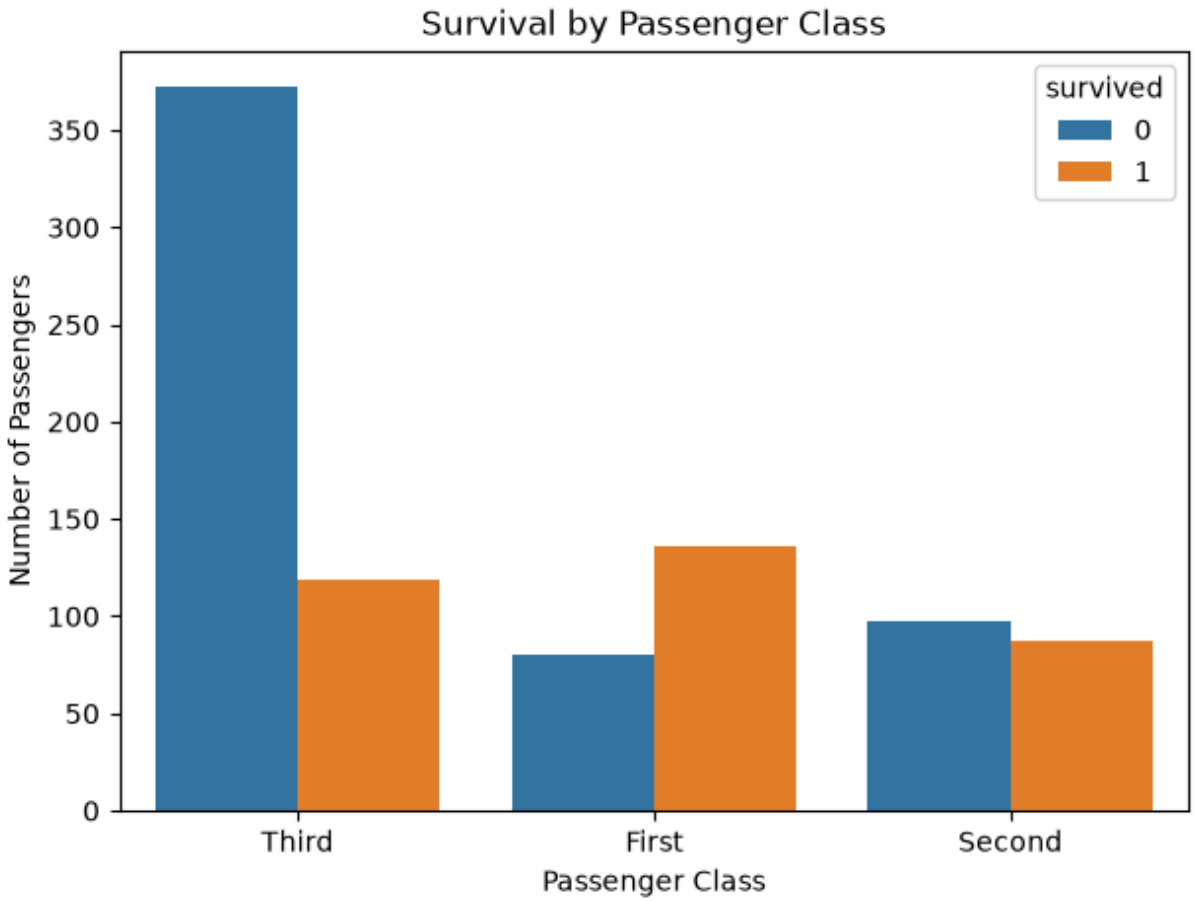
7.5 Survival by Passenger Class

In [17]:
```
plt.figure(figsize=(7, 5))

sns.countplot(data=df, x="class", hue="survived")

plt.title("Survival by Passenger Class")
plt.xlabel("Passenger Class")
plt.ylabel("Number of Passengers")

plt.show()
```



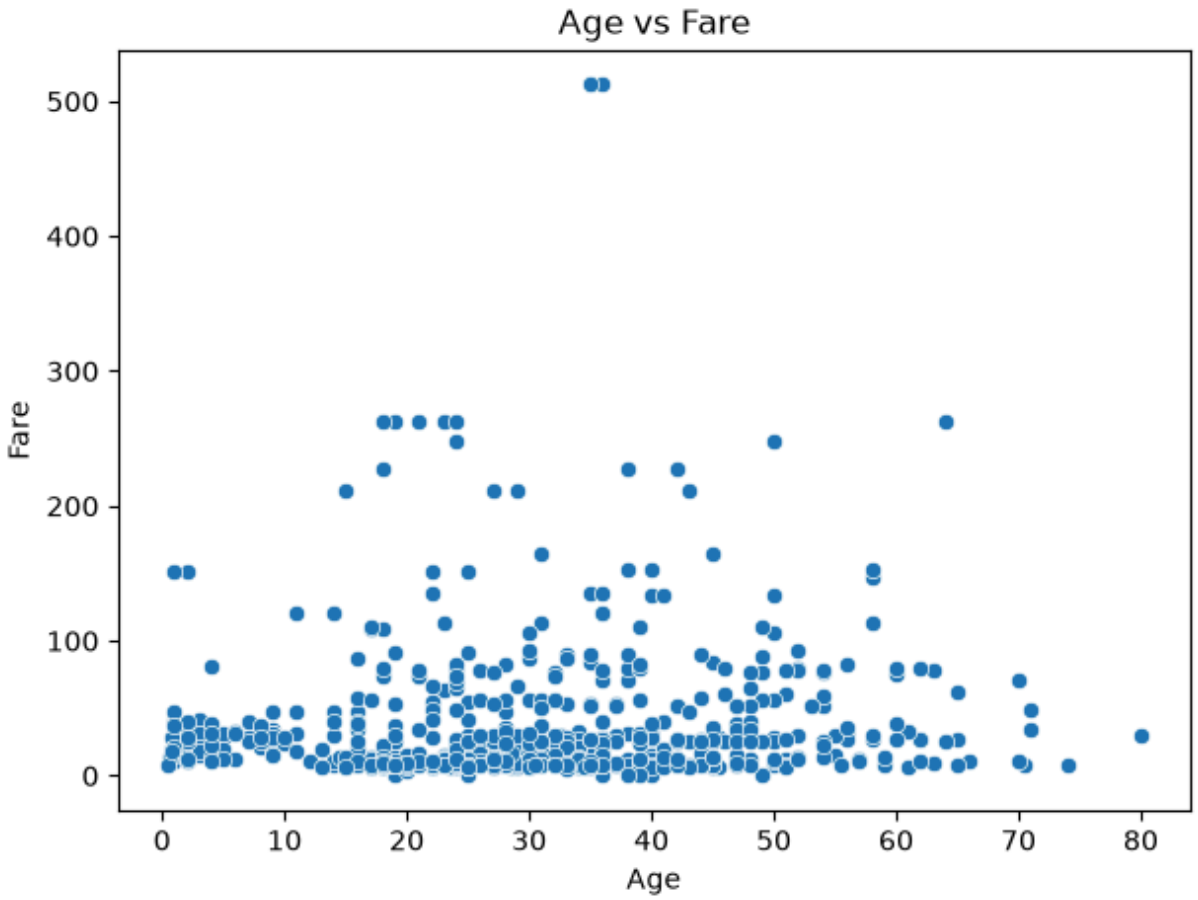
7.6 Age vs Fare (Scatter Plot)

In [18]:
```
plt.figure(figsize=(7, 5))

sns.scatterplot(data=df, x="age", y="fare")

plt.title("Age vs Fare")
plt.xlabel("Age")
plt.ylabel("Fare")

plt.show()
```



8. Summary and Conclusion

In [19]:
```
print("Total Passengers:", len(df))
print("Average Age:", df["age"].mean())
print("Average Fare:", df["fare"].mean())
print("Survival Rate:", df["survived"].mean())
print("Age-Fare Correlation:", df["age"].corr(df["fare"]))
```

```
Total Passengers: 891
Average Age: 29.69911764705882
Average Fare: 32.204207968574636
Survival Rate: 0.3838383838383838
Age-Fare Correlation: 0.09606669176903888
```